Inducción frontend

Proyecto Alojamientos

Guía completa antes de iniciar los tutoriales del frontend SPA vanilla

Elemento	Descripción
Stack	HTML5, CSS3, JavaScript vanilla, Vite, Bootstrap 5.3, Bootstrap Icons, Fetch API, npm, Git y Git Bash.
Público	Estudiantes que ya conocen HTML, CSS, JavaScript, Bootstrap, Python, backend, APIs, Git, SQL, MySQL y Scrum.
Propósito	Conectar lo aprendido en backend con la construcción del frontend del MVP.
Resultado esperado	Entrar al Tutorial 1 entendiendo arquitectura, flujo, carpetas, archivos, permisos, endpoints y validación.

Uso recomendado: Este documento no reemplaza los tutoriales. Sirve como inducción completa para entender qué se hará, por qué se hará y cómo se organiza el frontend antes de escribir código.

Contenido

- 1. Propósito de esta inducción
- 2. Punto de partida de los estudiantes
- 3. Qué se va a construir
- 4. Qué no se va a construir todavía
- 5. Stack del frontend
- 6. Relación con el backend
- 7. Conceptos clave antes de programar
- 8. Arquitectura general del proyecto
- 9. Mapa detallado de archivos
- 10. Flujo general de la SPA
- 11. Navegación final del MVP
- 12. Módulos del MVP
- 13. Permisos importantes
- 14. Errores HTTP que deben entender
- 15. Cómo se trabajará por tutoriales
- 16. Cómo se trabajará con Git
- 17. Validación del resultado
- 18. Errores comunes que deben evitar
- 19. Reglas de trabajo durante el frontend
- 20. Mini glosario operativo
- 21. Documentos de apoyo
- 22. Checklist antes de empezar
- 23. Cierre

1. Propósito de esta inducción

Este documento prepara al estudiante antes de empezar la construcción del frontend del proyecto Alojamientos. Su objetivo es ofrecer una visión completa del camino técnico, funcional y pedagógico que se seguirá durante los tutoriales.

- No reemplaza los tutoriales paso a paso.
- No reemplaza el contrato backend-frontend.
- No reemplaza la arquitectura completa del proyecto.
- Sirve para entender qué se va a construir, por qué se construye así y cómo se validará.

La idea principal es que el estudiante entre al Tutorial 1 con contexto suficiente para no copiar código de forma mecánica, sino entender el propósito de cada carpeta, archivo, ruta, formulario, servicio y validación.

2. Punto de partida de los estudiantes

El grupo no parte de cero absoluto. Ya conoce conceptos que serán reutilizados durante el frontend. Esta inducción no vuelve a enseñar HTML, CSS, JavaScript o backend desde cero; los conecta con el proyecto.

Conocimiento previo	Cómo se conecta con este frontend
HTML	Estructura de formularios, vistas, cards, tablas, navbar y contenedores.
CSS	Ajustes visuales globales y personalización mínima sobre Bootstrap.
JavaScript	Renderizado dinámico, eventos, módulos, manejo de sesión, consumo de API y navegación.
Bootstrap	Diseño de navbar, cards, tablas, formularios, badges, alertas y layout responsive.
Python y backend	Comprensión de cómo el frontend consume endpoints expuestos por el servidor.
APIs	Comunicación por HTTP, uso de JSON, status codes y payloads.
Git	Trabajo por commits, control de cambios y avance por sprints.
SQL y MySQL	Comprensión de los datos que el backend expone al frontend.
Scrum	Organización por sprints, entregables y validación incremental.

Nota: El reto ahora es pasar de construir el backend a construir la interfaz que lo consume. El frontend no reemplaza al backend: lo usa, muestra sus datos, envía formularios y respeta sus reglas de seguridad.

3. Qué se va a construir

Se construirá una SPA vanilla del proyecto Alojamientos. SPA significa Single Page Application: una aplicación que tiene una sola entrada HTML y cambia sus vistas de forma dinámica dentro del navegador.

- Una aplicación web en el navegador.
- Una sola entrada HTML con un punto de montaje #app.
- Navegación interna sin recargar toda la página.
- Consumo de un backend real mediante Fetch API.
- Registro, login, logout y perfil del usuario.
- Roles, permisos visuales y guards de navegación.

- Listado público de alojamientos desde Inicio.
- Detalle de alojamiento.
- Creación, edición y eliminación de alojamientos según permisos.
- Catálogo público de comodidades.
- Creación de comodidad solo para admin.
- Asignación de comodidades a alojamientos por propietario o admin.

Nota: Una decisión final del MVP es que Inicio muestra el listado público de alojamientos. Por eso la navbar no necesita un enlace adicional llamado Alojamientos.

4. Qué no se va a construir todavía

Hay módulos importantes que pertenecen al futuro del producto, pero no forman parte del MVP actual porque dependen de endpoints reales que todavía no están disponibles en el backend.

- Imágenes reales de alojamientos.
- Reservas.
- Búsqueda y disponibilidad real.
- Favoritos.
- Reseñas.
- Despliegue del frontend.

Alcance: Estos módulos no deben simularse con datos falsos ni con localStorage como si fueran funcionalidades reales. En el curso se documentan como futuros para entender el crecimiento posible del sistema sin mezclarlo con el MVP actual.

5. Stack del frontend

Herramienta	Uso en el proyecto
HTML5	Define la entrada base index.html y la estructura general que se renderiza.
CSS3	Permite estilos globales y ajustes visuales sobre Bootstrap.
JavaScript vanilla moderno	Controla componentes, páginas, router, eventos, sesión y consumo de API.
Vite	Servidor de desarrollo, soporte de módulos e imports, build de producción y preview del build compilado.
Bootstrap 5.3	Sistema visual para formularios, cards, tablas, navbar, alertas y layout.
Bootstrap Icons	Iconografía del proyecto.
Fetch API	Consumo HTTP del backend.
npm	Instalación de dependencias y ejecución de scripts.
Git	Control de versiones y commits por etapa.

npm run dev levanta el servidor de desarrollo, normalmente en http://localhost:5173/; npm run build genera dist/; npm run preview sirve el build compilado, normalmente en http://localhost:4173/. preview sirve para revisar el resultado compilado y no reemplaza dev.

6. Relación con el backend

El backend ya expone una API. El frontend la consume usando la URL base configurada en una variable de entorno local.

```
.env.example documenta la configuración esperada y sí se versiona. .env es local, se puede crear copiando .env.example, y no debe subirse al repositorio.
```

Las variables expuestas al frontend deben empezar por VITE_. La variable principal es VITE_API_URL=http://localhost:5000/api/v1. Desde Sprint 2 es necesaria cuando empieza el consumo real del backend. Los módulos API del frontend nunca deben inventar endpoints; deben usar el contrato aprobado y respetar los nombres técnicos exactos. Para validar npm run preview, el backend debe permitir por CORS el origen de preview, normalmente http://localhost:4173/.

Campo técnico	Uso correcto
contrasena	Campo técnico para contraseña en payloads de registro y login. No lleva ñ.
access_token	Token devuelto por el backend al iniciar sesión.
usuario_id	Identificador del propietario de un alojamiento.
precio_noche	Precio por noche del alojamiento.
comodidades_ids	Arreglo de IDs para asignar o vaciar comodidades de un alojamiento.

Seguridad: El backend valida definitivamente los permisos. El frontend puede ocultar botones o bloquear vistas por experiencia de usuario, pero esa protección visual no reemplaza la seguridad real del servidor.

7. Conceptos clave antes de programar

Concepto	Explicación práctica
SPA	Aplicación con una sola entrada HTML que cambia vistas dentro de #app.
Página	Vista completa, por ejemplo Login, Perfil, Usuarios, Alojamientos o Comodidades.
Componente	Pieza reutilizable de interfaz, por ejemplo Navbar, AlojamientoCard o ComodidadBadge.
Servicio API	Archivo que llama al backend sin renderizar HTML.
Cliente HTTP	Módulo central que maneja base URL, JSON, token, 204 y errores visibles.
Sesión	Token y usuario guardados en el frontend para mantener autenticación.
Autenticación	Proceso para saber quién es el usuario.
Autorización	Proceso para saber qué puede hacer el usuario.
Guards	Funciones que protegen rutas privadas o solo admin, a partir de Sprint 3.
Estados de interfaz	loading, error, empty, success, unauthorized y forbidden.

7.1 Conceptos con más detalle

SPA

La SPA evita tener una página HTML distinta para cada pantalla. En este proyecto, index.html contiene el contenedor #app, y JavaScript decide qué vista mostrar. Cuando el usuario navega, no se recarga toda la página; se reemplaza el contenido del contenedor.

Página

Una página representa una vista completa. Por ejemplo, LoginPage muestra el formulario de login, PerfilPage muestra los datos del usuario, y AlojamientosPage muestra el catálogo principal. Las páginas pueden renderizar HTML, registrar eventos y cargar datos.

Componente

Un componente es una pieza reutilizable. No debe saber todo del proyecto. AlojamientoCard recibe datos de un alojamiento y devuelve HTML para mostrarlo. ComodidadBadge recibe una comodidad y la muestra como etiqueta. La responsabilidad del componente debe ser pequeña y clara.

Servicio API

Un servicio API centraliza llamadas a endpoints de un recurso. auth.api.js conoce endpoints de usuarios, alojamientos.api.js conoce endpoints de alojamientos y comodidades.api.js conoce endpoints de comodidades. Estos archivos no renderizan HTML.

Cliente HTTP

http.js es el punto común para hacer peticiones. Ahí se agrega la URL base, Content-Type, Authorization con token, manejo de JSON, manejo de 204 y normalización de errores. Los errores visibles se apoyan en src/js/utils/errorMessages.js. Esto evita repetir fetch en todas las páginas.

Guards

Los guards forman parte de la arquitectura final; en Sprint 2 todavía no existen como archivos separados porque la navegación simple sigue viviendo en main.js.

8. Arquitectura general del proyecto

La estructura se organiza por responsabilidades para evitar que todo quede mezclado en main.js cuando la aplicación ya crece. En Sprint 1 y Sprint 2, main.js concentra la base y una navegación simple; después esa responsabilidad se separa en router y guards. Primero se muestra una visión general y luego un mapa detallado de archivos.

```
frontend-alojamientos/ incluye index.html, package.json, package-lock.json, .env.example,
vite.config.js y la carpeta src/ con css/main.css y js/main.js.
```

Carpeta	Responsabilidad
api/	Comunicación con backend por recurso.
auth/	Sesión, usuario actual y permisos.

router/	Rutas, parámetros y guards a partir de Sprint 3.
components/	Piezas reutilizables de interfaz.
pages/	Vistas completas de la SPA.
utils/	Funciones auxiliares como formato de moneda, almacenamiento local y mensajes de error.
css/	Estilos base del proyecto.

9. Mapa detallado de archivos

Esta sección muestra qué hace cada archivo importante del frontend y en qué momento aparece dentro de los tutoriales. No es necesario memorizarlo; sirve como mapa de lectura antes de empezar.

9.1 Archivos raíz

Archivo	Aparece en	Qué hace
index.html	Tutorial 1	Entrada única de la SPA. Contiene #app y carga src/js/main.js.
package.json	Tutorial 1	Define dependencias y scripts dev, build y preview.
.env.example	Tutorial 1	Documenta VITE_API_URL y la configuración esperada del frontend.
vite.config.js	Tutorial 1	Configuración de Vite, normalmente mínima.

9.2 CSS y arranque

Archivo	Aparece en	Qué hace
src/css/main.css	Tutorial 1	Estilos base del proyecto y ajustes globales.
src/js/main.js	Tutorial 1 a 5	Arranca la SPA: monta la pantalla base en Sprint 1, coordina la navegación simple en Sprint 2 y después se integra con router y guards.

9.3 API

Archivo	Aparece en	Qué hace
src/js/api/http.js	Tutorial 2	Cliente HTTP centralizado: base URL, headers, token, JSON, 204 y errores visibles apoyados en errorMessages.js.
src/js/api/auth.api.js	Tutorial 2	Registro, login, obtener perfil y actualizar perfil.
src/js/api/admin.api.js	Tutorial 3	Consume GET /admin/usuarios.
src/js/api/alojamientos.api.js	Tutorial 4	Lista, detalla, crea, actualiza y elimina alojamientos.
src/js/api/comodidades.api.js	Tutorial 5	Lista, detalla, crea comodidades y actualiza comodidades de un alojamiento.

9.4 Autenticación y permisos

Archivo	Aparece en	Qué hace
src/js/auth/session.service.js	Tutorial 2	Guarda, lee y limpia token y usuario. Centraliza la sesión.
src/js/auth/permissions.js	Tutorial 3 y 4	Indica si hay sesión, si es admin y si puede gestionar un alojamiento.

9.5 Router

Archivo	Aparece en	Qué hace
src/js/router/routes.js	Tutorial 3 a 5	Define rutas, páginas, permisos, parámetros y hooks de montaje.
src/js/router/guards.js	Tutorial 3	Contiene requireAuth y requireAdmin para proteger rutas.

9.6 Componentes

Archivo	Aparece en	Qué hace
src/js/components/layout/Navbar.js	Tutorial 2 y 3	Navbar pública y autenticada. En Sprint 2 muestra el correo y el acceso a Perfil y Cerrar sesión; desde Sprint 3 agrega Usuarios para admin.
src/js/components/alojamientos/AlojamientoCard.js	Tutorial 4	Card del listado de alojamientos. Muestra datos básicos y Ver detalle.
src/js/components/alojamientos/AlojamientoForm.js	Tutorial 4	Formulario reutilizable para crear y editar alojamientos.
src/js/components/alojamientos/ComodidadesSelector.js	Tutorial 5	Checkboxes para seleccionar comodidades y generar comodidades_ids.
src/js/components/comodidades/ComodidadBadge.js	Tutorial 5	Muestra una comodidad como badge o etiqueta visual.

9.7 Páginas

Archivo	Aparece en	Qué hace
LoginPage.js	Tutorial 2	Formulario de login. Guarda token y usuario. Login exitoso va a Inicio.
RegistroPage.js	Tutorial 2	Formulario de registro. Registro exitoso va a Login.
PerfilPage.js	Tutorial 2	Muestra el correo como dato visible, carga datos reales del perfil y permite actualizar nombre, apellido y telefono.
AdminUsuariosPage.js	Tutorial 3	Lista usuarios para admin desde GET /admin/usuarios.
AlojamientosPage.js	Tutorial 4	Listado público de alojamientos. Inicio usa esta página.
AlojamientoDetallePage.js	Tutorial 4	Detalle de alojamiento y acciones Editar/Eliminar cuando corresponda.
CrearAlojamientoPage.js	Tutorial 4	Formulario para crear alojamiento autenticado.
EditarAlojamientoPage.js	Tutorial 4 y 5	Edita alojamiento y luego integra selector de comodidades.
ComodidadesPage.js	Tutorial 5	Catálogo público de comodidades.
ComodidadDetallePage.js	Tutorial 5	Detalle público de una comodidad.
AdminCrearComodidadPage.js	Tutorial 5	Formulario solo admin para crear comodidad.

9.8 Utilidades

Archivo	Aparece en	Qué hace
src/js/utils/storage.js	Tutorial 2	Lectura, escritura y eliminación de datos en almacenamiento local.
src/js/utils/formatters.js	Tutorial 4	Formato de moneda, fechas o textos cortos.

src/js/utils/errorMessages.js	Tutorial 2	Normaliza o traduce mensajes visibles del backend al español.

10. Flujo general de la SPA

1. El usuario abre la aplicación en el navegador.
2. index.html carga src/js/main.js.
3. main.js monta la SPA dentro de #app.
4. En Sprint 1, main.js muestra una pantalla base temporal.
5. En Sprint 2, main.js mantiene una navegación simple para usuarios y autenticación.
6. Desde Sprint 3, el router identifica qué página debe mostrarse y los guards validan acceso.
7. Se enlazan eventos después de insertar el HTML.
8. Se cargan datos desde el backend si la página los necesita.
9. La UI muestra loading, datos, error, empty o success según el resultado.

```
render -> bindEvents -> cargarDatos
```

Nota: Los eventos deben registrarse después de que el HTML exista en el DOM. Registrar eventos antes del renderizado es una causa común de errores.

10.1 Flujo de una petición al backend

10. Una página necesita datos o envía un formulario.
11. La página llama a un servicio API, por ejemplo alojamientos.api.js.
12. El servicio API llama a request() en http.js.
13. http.js agrega VITE_API_URL, headers JSON y token si existe.
14. El backend responde con data o con error.
15. http.js normaliza la respuesta o el error.
16. La página actualiza la interfaz con datos, éxito o mensaje de error.

11. Navegación final del MVP

Perfil	Navbar final
Visitante	Inicio, Comodidades, Login, Registro
Usuario autenticado	Inicio, Comodidades, dropdown con el correo del usuario
Admin	Inicio, Comodidades, Usuarios, dropdown con el correo del usuario

El dropdown del usuario autenticado contiene Perfil, una línea divisoria y Cerrar sesión. La opción Usuarios aparece solo si el usuario tiene rol admin. No hay panel intermedio de administración: Usuarios abre directamente el listado de usuarios.

12. Módulos del MVP

12.1 Usuarios y autenticación

El módulo de usuarios permite registrar, iniciar sesión, cerrar sesión y editar datos básicos del perfil.

Endpoint	Uso en frontend
POST /usuarios/registro	Crear usuario. Registro exitoso navega a Login.
POST /usuarios/login	Iniciar sesión. Login exitoso guarda token y navega a Inicio.
GET /usuarios/perfil	Consultar perfil autenticado.
PATCH /usuarios/perfil	Actualizar nombre, apellido y teléfono.

Muestra el correo como dato visible, carga datos reales del perfil y permite actualizar nombre, apellido y telefono.

12.2 Roles y usuarios admin

El frontend maneja roles para mejorar la experiencia visual y proteger rutas, pero el backend valida permisos reales.

Elemento	Comportamiento
Usuarios en navbar	Visible solo para admin.
GET /admin/usuarios	Lista usuarios para admin.
401	Puede limpiar sesión y pedir login.
403	Muestra permisos insuficientes sin cerrar sesión.

12.3 Alojamientos

Inicio muestra el listado público de alojamientos. Las cards del listado muestran solo Ver detalle. Las acciones Editar y Eliminar viven en el detalle cuando el usuario es propietario o admin.

Endpoint	Uso en frontend
GET /alojamientos	Listado público en Inicio.
GET /alojamientos/:id	Detalle público.
POST /alojamientos	Crear alojamiento con token.
PATCH /alojamientos/:id	Editar alojamiento propio o como admin.
DELETE /alojamientos/:id	Eliminar alojamiento propio o como admin. Devuelve 204.

Los textos de botones son simples: Volver, Crear y Actualizar.

12.4 Comodidades

Las comodidades son públicas para consulta. Crear comodidad es una acción solo admin. Asignarlas a un alojamiento corresponde al propietario o al admin.

Endpoint	Uso en frontend
GET /comodidades	Catálogo público.
GET /comodidades/:id	Detalle público.
POST /comodidades	Crear comodidad solo como admin.
PATCH /alojamientos/:id/comodidades	Asignar o vaciar comodidades con comodidades_ids.

```
{
  "comodidades_ids": [1, 2, 3]
}

{
  "comodidades_ids": []
}
```

El botón principal en Crear comodidad se llama Guardar. No se implementa editar ni eliminar comodidad.

13. Permisos importantes

Acción	Visitante	Usuario	Propietario	Admin
Ver alojamientos	Sí	Sí	Sí	Sí
Ver detalle de alojamiento	Sí	Sí	Sí	Sí
Crear alojamiento	No	Sí	Sí	Sí
Editar alojamiento	No	No	Sí	Sí
Eliminar alojamiento	No	No	Sí	Sí
Ver comodidades	Sí	Sí	Sí	Sí
Crear comodidad	No	No	No	Sí
Asignar comodidades	No	No	Sí	Sí
Ver usuarios	No	No	No	Sí

La columna Propietario aplica cuando el usuario autenticado es dueño del alojamiento según usuario_id.

14. Errores HTTP que deben entender

Código	Significado	Comportamiento esperado
400	Datos inválidos	Mostrar validaciones en español y permanecer en la vista.
401	No hay token, token inválido o sesión expirada	Limpiar sesión o pedir login.
403	Usuario autenticado sin permiso	Mostrar permisos insuficientes sin cerrar sesión.
404	Recurso no encontrado	Mostrar mensaje de no encontrado.
500	Error del servidor	Mostrar error general en español.
204	Operación exitosa sin contenido	No intentar leer JSON; devolver null.

15. Cómo se trabajará por tutoriales

Tutorial	Resultado esperado
Tutorial 1	Base SPA con Vite, Bootstrap, package-lock.json, vite.config.js, #app, main.js, main.css y .env.example.
Tutorial 2	Usuarios, registro, login, sesión, perfil, navbar con dropdown y logout sobre la base creada en Tutorial 1.
Tutorial 3	Roles, guards y Usuarios para admin.
Tutorial 4	Alojamientos e Inicio como listado público.
Tutorial 5	Comodidades, creación admin y asignación a alojamientos.

Cada tutorial parte del anterior y agrega una capa nueva. La progresión debe sentirse natural: no son parches, son incrementos del MVP.

15.1 Qué debe lograr el estudiante al terminar cada tutorial

Tutorial	El estudiante debe poder explicar
1	Qué es Vite, dónde está #app, cómo se importa Bootstrap y cómo se valida dev/build/preview.
2	Cómo se guarda la sesión, cómo se consume login/registro y cómo funciona la navbar autenticada y el perfil.
3	Qué diferencia hay entre autenticación y autorización, y cómo se protegen rutas con guards.
4	Cómo una página consume datos, cómo se muestra un listado y

	cómo se gestionan permisos de propietario/admin.
5	Cómo se asignan relaciones con comodidades_ids y cómo se integran datos relacionados en una vista.

16. Cómo se trabajará con Git

Cada sprint debe cerrarse con una validación y un commit claro. El flujo básico es git status, npm run build, git add ., git commit -m "Sprint X: descripcion" y git push.

```
En los dos primeros tutoriales, los commits aprobados son Sprint 1: base SPA con Vite y
Bootstrap y Sprint 2: usuarios y autenticacion.
```

Etapa	Commit sugerido
Tutorial 1	Sprint 1: base SPA con Vite y Bootstrap
Tutorial 2	Sprint 2: usuarios y autenticacion
Tutorial 3	feat: implementar roles y guards
Tutorial 4	feat: implementar alojamientos frontend
Tutorial 5	feat: implementar comodidades frontend

17. Cómo se validará el resultado

La validación se realiza manualmente por perfil de usuario y por módulo funcional.

Perfil	Validaciones principales
Visitante	Ve Inicio, alojamientos, detalle, comodidades, login y registro. No gestiona recursos.
Usuario	Inicia sesión, edita perfil, crea alojamientos y gestiona sus propios alojamientos.
Propietario	Edita, elimina y asigna comodidades a sus alojamientos.
Admin	Ve usuarios, crea comodidades, gestiona alojamientos y asigna comodidades.

También se validan loading, empty, error, success, 401, 403 y 204 No Content.

18. Errores comunes que deben evitar

- Cambiar el campo técnico contrasena por contraseña en payloads.
- Leer el token fuera de response.data.access_token.
- Hacer fetch directo desde cualquier página sin usar el cliente HTTP.
- Guardar token en varios lugares.
- Confundir 401 con 403.
- Intentar leer JSON después de una respuesta 204.
- Poner Editar o Eliminar en las cards del listado de alojamientos.
- Dejar rutas o vistas legacy sin uso.
- Simular módulos futuros con localStorage.

- Inventar endpoints.
- Mantener demasiadas responsabilidades en main.js una vez que router y guards ya se separaron; en Sprint 2 la navegación simple en main.js es intencional.

19. Reglas de trabajo durante el frontend

- Seguir los tutoriales en orden.
- No saltarse sprints.
- No inventar endpoints.
- No modificar backend para resolver problemas de frontend.
- No agregar funcionalidades futuras.
- Mantener nombres técnicos exactos.
- Probar cada sprint antes de continuar.
- Ejecutar npm run build con frecuencia.
- Mantener la interfaz en español.
- Evitar textos con codificación dañada.
- Hacer commits pequeños y claros.

20. Mini glosario operativo

Término	Significado en este proyecto
#app	Contenedor donde JavaScript monta las vistas de la SPA.
route.params	Datos dinámicos de una ruta, por ejemplo el id de un alojamiento.
mount	Función que se ejecuta después de renderizar una página para enlazar eventos o cargar datos.
payload	Objeto JSON que se envía al backend.
response.data	Ubicación donde se leen los datos útiles de una respuesta exitosa.
token	Credencial enviada en Authorization para acceder a rutas protegidas.
owner/propietario	Usuario cuyo id coincide con usuario_id del alojamiento.
empty state	Mensaje o vista cuando una lista no tiene datos.
loading state	Mensaje o vista mientras se espera respuesta del backend.
forbidden	Estado en el que el usuario tiene sesión pero no permisos.

21. Documentos de apoyo

Documento	Cuándo consultarlo
CONCEPTOS-FUNDAMENTALES-FRONTEND.md	Para repasar conceptos antes y durante el desarrollo.
ARQUITECTURA-SPA-VANILLA.md	Para entender la organización del código.
CONTRATO-BACKEND-FRONTEND.md	Para validar endpoints, payloads y respuestas.
roadmap/roadmap-frontend.md	Para entender los 10 sprints y el alcance actual.
VALIDACION-MANUAL-FRONTEND.md	Para probar el MVP completo.
tutoriales/	Para construir paso a paso.
sprints/	Para entender objetivos, criterios y entregables.
anexos/	Para contexto complementario posterior.

22. Checklist antes de empezar

- Node.js instalado.
- npm instalado.
- Git instalado.
- Backend corriendo en http://localhost:5000/api/v1.
- Base de datos disponible.
- VITE_API_URL identificado.
- Editor de código listo.
- Terminal de Git Bash lista.
- Navegador y consola disponibles.
- Repositorio limpio o con cambios controlados.
- Tutorial 1 abierto.

23. Cierre

Esta inducción conecta lo que el estudiante ya sabe de backend, APIs y desarrollo web con la construcción de una SPA vanilla profesional. El objetivo no es memorizar archivos, sino entender la responsabilidad de cada parte y avanzar de forma ordenada.

Al terminar esta lectura, el siguiente paso es iniciar el Tutorial 1 y construir la base técnica del frontend. Desde ahí, cada tutorial agregará una capa funcional hasta completar el MVP implementable.